Assignment 2: Gaussian Classifier, Bias-Variance Decomposition, Evaluation Measures

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Automatic Testing Guidelines

Automatic unittesting requires you to submit a notebook which contains strictly defined objects. Strictness of definition consists of unified shapes, dtypes, variable names and more.

Within the notebook, we provide detailed instruction which you should follow in order to maximise your final grade.

Name your notebook properly, follow the pattern in the template name:

Assignment_N_NameSurname_matrnumber

1. N - number of assignment
2. NameSurname - your full name where every part of the name starts with a capital letter, no spaces
3. matrnumber - you student number on ID card (with k, potenitially with a leading zero)

Don't add any cells but use the ones provided by us. All cells have a unique ID so that the unit test can find it, so please do not add or remove any cell!

Always make sure that implemented functions have the correct output and given variables contain the correct data type. In the descriptions for every function you can find information on what datatype an output should have and you should stick to that in order to minimize conflicts with the unittest. Don't import any other packages than listed in the cell with the "imports" tag.

Questions are usually multiple choice (except the task description says otherwise) and can be answered by changing the given variables to either "True" or "False". "None" is counted as a wrong answer in any case!

Note: Never use variables you defined in another cell in your functions directly; always pass them to the function as a parameter. In the unitest, they won't be available either. If you want to make sure that everything is executable for the unittest, try executing cells/functions individually (instead of running the whole notebook).

Task 1: Gaussian classifier: visualization & parameter estimation

The goal of this task is to explore the given (artificial) data before diving into the classification function. To do this, we will use `matplotlib` to plot the data set and `numpy` to estimate the means & covariance matrices of the classes as well as the probability of encountering a positive/negative example.

- **Plot 1.1**: Visualize the data stored in `normal.csv` with two different colors using a scatter plot and store it in the given variable. Always label the axes of all your plots. We also suggest to make a plot legend indicating which color belongs to which label.
- **Question 1.1**: Answer 2 questions regarding the data depicted in the plot.
- **Code 1.2**: We assume that the data is distributed according to a two-dimensional (bivariate) normal distribution:
  - Write a function that estimates the mean and covariance matrix for the entire dataset, the means and covariance matrices for each class, and the probabilities $p(y = +1)$ and $p(y = -1)$.
  - Return a tuple containing the results (the resulting list should be of length 8). The datatype for `meanX`, `covX`, `meanXpos`, `covXpos`, `meanXneg`, and `covXneg` should be a numpy array, for $p(y = +1)$ and $p(y = -1)$ it should be float.

Plot 1.1 (2 Points):

In [1]:
```
# Nothing to do here, just run the cell.
import sklearn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

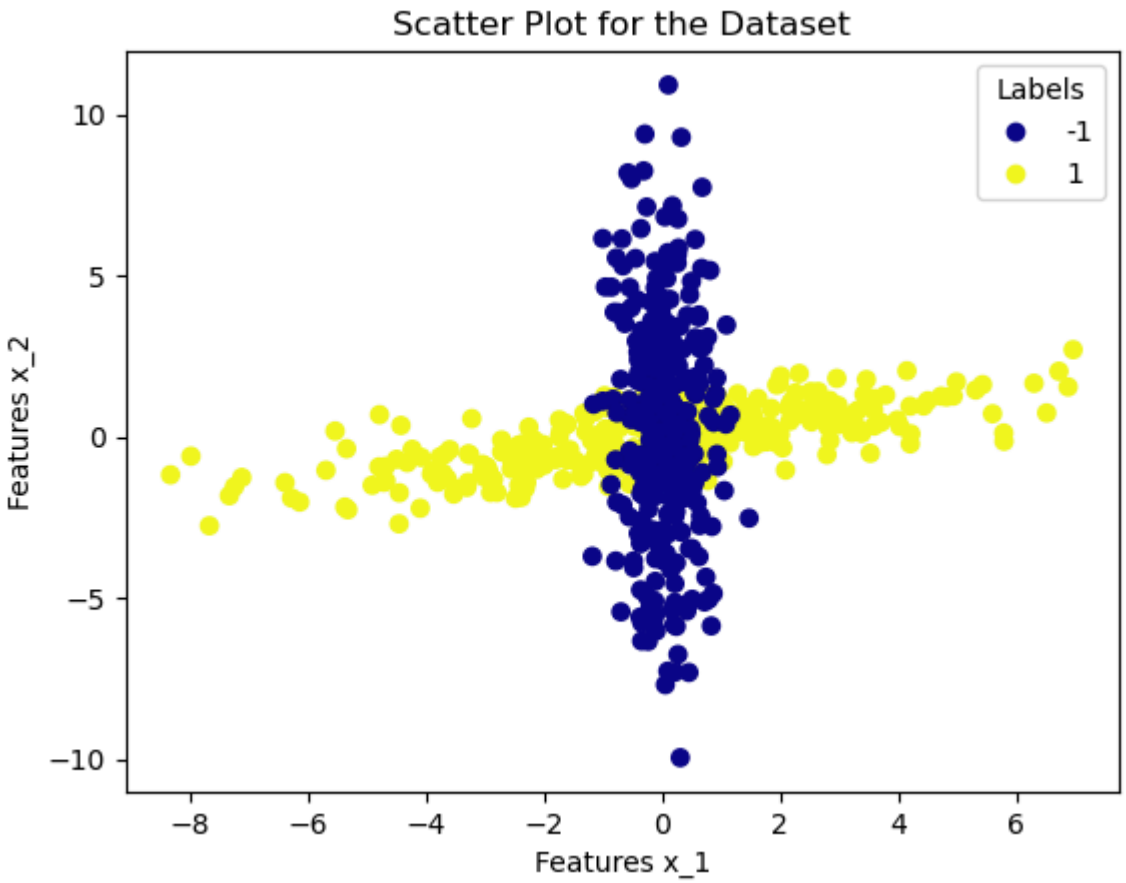
In [2]:
```
# Nothing to do here, just run the cell.
Z = np.genfromtxt('normal.csv', delimiter=',')
X, y = Z[:,:-1].astype(np.float32), Z[:,-1].astype(int)
```

In [83]:
```
def scatter_plot(X: np.ndarray, y: np.ndarray):
    """Creates a scatter-plot for the dataset X with labels y.

    Parameters
    ----------
    X : (N, K) np.ndarray
        Data to plot.
    y : (N,) np.ndarray
        labels of the data.
    """
    scatter = plt.scatter(X[:,0], X[:,1], c=y, cmap="plasma")
    plt.legend(handles=scatter.legend_elements()[0], labels=["-1", "1"],
    plt.xlabel("Features x_1")
```

```
    plt.ylabel("Features x_2")
    plt.title("Scatter Plot for the Dataset")
    plt.show()
```

In [84]:
```
# Nothing to do here, just run the cell.
scatter_plot(X, y)
```



Question 1.1 (2 Points):

Answer the following yes/no questions concerning the distribution of the data:

a1_) Would a linear regression method be an optimal choice for this task?
b1_) Would a linear classifier achieve a better performance than 25% misclassification?

To answer the question, assign "True" or "False" boolean values to variables in the next cell. For example, if you think that **a1_)** is correct, define a variable `a1_` and set it to `True`, the same applies to **b1_)**. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.
Note: Do not reuse these variable names. They are used for testing.

In [5]:
```
# YOUR CODE HERE
a1_ = False
b1_ = False
```

In [6]:
```
# DO NOT DELETE THIS CELL!
assert a1_ is not None, "Store True/False!"
assert a1_ in [True, False], "Invalid Answer!"
```

In [7]:
```
# DO NOT DELETE THIS CELL!
assert b1_ is not None, "Store True/False!"
assert b1_ in [True, False], "Invalid Answer!"
```

Code 1.2 (6 Points):

In [8]:
```
def est_mean_cov(X: np.ndarray, y: np.ndarray) -> tuple[np.ndarray, np.nd
    """Estimate means and covariance matrices from the given data, as wel
    Utilize numpy functions for calculating the results.

    Parameters
    ----------
    X : (N, K) np.ndarray
        Data matrix of shape (n_samples, n_features).
    y : (N,) np.ndarray
        Data vector of shape (n_samples,), with binary labels (e.g., +1 o

    Returns
    -------
    meanX : (K,) np.ndarray
        Mean vector (one entry per feature) for the entire dataset.
    covX : (K, K) np.ndarray
        Covariance matrix for the entire dataset.
    meanXpos : (K,) np.ndarray
        Mean vector (one entry per feature) for positive samples.
    covXpos : (K, K) np.ndarray
        Covariance matrix for positive samples.
    meanXneg : (K,) np.ndarray
        Mean vector (one entry per feature) for negative samples.
    covXneg : (K, K) np.ndarray
        Covariance matrix for negative samples.
    p_ypos : float
        Probability of a positive example, p(y = +1).
    p_yneg : float
        Probability of a negative example, p(y = -1).
    """
    # YOUR CODE HERE

    meanX = np.mean(X, axis=0)
    covX = np.cov(X, rowvar=False)
    Xpos = X[y==1]
    Xneg = X[y==-1]
    mean_Xpos = np.mean(Xpos, axis=0)
    mean_Xneg = np.mean(Xneg, axis=0)
    covXpos = np.cov(Xpos, rowvar=False)
    covXneg = np.cov(Xneg, rowvar=False)
    p_ypos = np.mean(y==1)
    p_yneg = np.mean(y==-1)

    return meanX, covX, mean_Xpos, covXpos, mean_Xneg, covXneg, p_ypos, p
```

In [9]:
```
# DO NOT DELETE THIS CELL!
res = est_mean_cov(X, y)
assert len(res) == 8, "The number of outputs is wrong!"
meanX, covX, meanXpos, covXpos, meanXneg, covXneg, p_ypos, p_yneg = res
```

In [10]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(meanX, np.ndarray), "The mean of the whole data is not
np.testing.assert_array_almost_equal(meanX, np.array([-0.057087306,  0.06
```

In [11]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(covX, np.ndarray), "The covariance of the whole data is
np.testing.assert_array_almost_equal(covX, np.array([[4.321541110644044,
```

In [12]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(meanXpos, np.ndarray), "The mean of the positive sample
np.testing.assert_array_almost_equal(meanXpos, np.array([-0.14820707, -0.
```

In [13]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(covXpos, np.ndarray), "The covariance of the positive s
np.testing.assert_array_almost_equal(covXpos, np.array([[8.44226725, 2.01
```

In [14]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(meanXneg, np.ndarray), "The mean of the negative sample
np.testing.assert_array_almost_equal(meanXneg, np.array([0.03403238, 0.16
```

In [15]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(covXneg, np.ndarray), "The covariance of the negative s
np.testing.assert_array_almost_equal(covXneg, np.array([[ 0.19860714, -0.
```

In [16]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(p_ypos, float), "The probably of a positive sample is n
assert p_ypos == 0.5, "The probability of a positive sample is not correc
```

In [17]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(p_yneg, float), "The probability of a negative sample i
assert p_yneg == 0.5, "The probability of a negative sample is not correc
```

In [18]:
```
# Nothing to do here, just run the cell.
print("Entire dataset:\n")
print("Mean = ", meanX, "\n")
print("Covariance:")
print(pd.DataFrame(covX,columns=["x1","x2"],index=["x1","x2"]),"\n")
print(30 * "=")
print("Positive class:\n")
print("Mean = ", meanXpos, "\n")
print("Covariance:")
print(pd.DataFrame(covXpos,columns=["x1","x2"],index=["x1","x2"]),"\n")
print("p(y=+1) =", p_ypos, "\n")
print(30 * "=")
print("Negative class:\n")
print("Mean =", meanXneg, "\n")
print("Covariance:")
print(pd.DataFrame(covXneg,columns=["x1","x2"],index=["x1","x2"]),"\n")
print("p(y=-1) =", p_yneg)
```

```
Entire dataset:

Mean =  [-0.05708731  0.06468879]

Covariance:
          x1        x2
x1  4.321541  0.936189
x2  0.936189  6.752614

==============================
Positive class:

Mean =  [-0.14820707 -0.03790113]

Covariance:
          x1        x2
x1  8.442267  2.017815
x2  2.017815  0.953326

p(y=+1) = 0.5

==============================
Negative class:

Mean = [0.03403238 0.16727877]

Covariance:
          x1         x2
x1  0.198607  -0.161064
x2 -0.161064  12.553366

p(y=-1) = 0.5
```

Task 2: Gaussian classifier: compute classifier & visualization

Now that we got a good idea of the data, we want to implement a classifier and visualize it.

- **Code 2.1**: Compute an optimal classification function g_opt in `calc_g_opt()`. To do this, you should:
  - Calculate the values of the corresponding parameters $\mathbf{A}$, $\mathbf{b}$ and c in the provided functions (have a look at the lecture slides).
  - Store the results in the given parameters `A` (np.ndarray), `b` (np.ndarray), `c` (float), and `g_opt` (np.ndarray).
  - Print the values of $\mathbf{A}$, $\mathbf{b}$ and c that you have calculated with their respective shapes.
  - Note: You can reuse items from the previous task.

- **Plot 2.2**: Visualize the classification function and the original data samples from Task 1.1. in **one** two-dimensional plot. I.e., the plot should show the data samples -- blue for positive and orange for negative -- and also show the classification function (500x500 grid points) in the corresponding (or similar) colors. For the grid points: use alpha=0.05 for opaquness and s=1 for the marker size. The decision

boundary will become visible as the line separating the two classification territories. Again: Label the axes and plot a legend.

- **Question 2.2**: Answer 2 questions about the result.

Code 2.1 (10 Points):

In [ ]:
```
def calc_A(covXpos: np.ndarray, covXneg: np.ndarray) -> np.ndarray:
    """Calculate the desired parameter A based on covariance matrices and

    Parameters
    ----------
    covXpos : (K, K) np.ndarray
        Covariance matrix of positive samples.
    covXneg : (K, K) np.ndarray
        Covariance matrix of negative samples.

    Returns
    -------
    A : (K, K) np.ndarray
        Result of the parameter calculation.
    """
    # YOUR CODE HERE
    inv_covXpos = np.linalg.inv(covXpos)
    inv_covXneg = np.linalg.inv(covXneg)
    A = inv_covXpos-inv_covXneg

    return A
```

In [20]:
```
# DO NOT DELETE THIS CELL!
A = calc_A(
    np.array([[8.44226725, 2.01781502], [2.01781502, 0.95332618]], dtype=
    np.array([[ 0.19860714, -0.16106351], [-0.16106351, 12.55336585]], dt
)
assert isinstance(A, np.ndarray), "Parameter A is not a np.ndarray!"
assert A.shape == (2, 2), "Parameter A does not have the correct shape!"
np.testing.assert_array_almost_equal(A, np.array([[-4.8482757, -0.5726957
```

In [21]:
```
def calc_b(meanXpos: np.ndarray, covXpos: np.ndarray, meanXneg: np.ndarra
    """This function should contain the calculations for the respective b

    Parameters
    ----------
    meanXpos : (K,) np.ndarray
        Mean vector of positive samples.
    covXpos : (K, K) np.ndarray
        Covariance matrix of positive samples.
    meanXneg : (K,) np.ndarray
        Mean vector of negative samples.
    covXneg : (K, K) np.ndarray
        Covariance matrix of negative samples.

    Returns
    -------
    b : (K,) np.ndarray
        Result of the parameter calculation.
    """
    # YOUR CODE HERE
```

```
    inv_covXpos = np.linalg.inv(covXpos)
    inv_covXneg = np.linalg.inv(covXneg)
    b = np.dot(inv_covXpos,meanXpos)-np.dot(inv_covXneg,meanXneg)

    return b
```

In [22]:
```
# DO NOT DELETE THIS CELL!
b = calc_b(
    np.array([-0.14820707, -0.037901126], dtype=np.float32),
    np.array([[8.44226725, 2.01781502], [2.01781502, 0.95332618]], dtype=
    np.array([0.03403238, 0.16727877], dtype=np.float32),
    np.array([[ 0.19860714, -0.16106351], [-0.16106351, 12.55336585]], dt
)
assert isinstance(b, np.ndarray), "Parameter b is not a np.ndarray!"
assert b.shape == (2,), "Parameter b does not have the correct shape!"
np.testing.assert_array_almost_equal(b, np.array([-0.2003752 , -0.0209471
```

In [23]:
```
def calc_c(meanXpos: np.ndarray, covXpos: np.ndarray, meanXneg: np.ndarra
    """This function should contain the calculations for the respective c

    Parameters
    ----------
    meanXpos : (K,) np.ndarray
        Mean vector of positive samples.
    covXpos : (K, K) np.ndarray
        Covariance matrix of positive samples.
    meanXneg : (K,) np.ndarray
        Mean vector of negative samples.
    covXneg : (K, K) np.ndarray
        Covariance matrix of negative samples.
    p_ypos : float
        Probability of encountering a positive sample.
    p_yneg : float
        Probability of encountering a negative sample.

    Returns
    -------
    c : float
        Result of the parameter calculation.
    """
    # YOUR CODE HERE
    #Unit2 slides page 22
    inv_covXpos = np.linalg.inv(covXpos)
    inv_covXneg = np.linalg.inv(covXneg)

    term1 = -0.5 * (meanXpos.T @ inv_covXpos @ meanXpos)
    term2 = 0.5 * (meanXneg.T @ inv_covXneg @ meanXneg)
    term3 = -0.5 * np.log(np.linalg.det(covXpos))
    term4 = 0.5 * np.log(np.linalg.det(covXneg))
    term5 = np.log(p_ypos)
    term6 = -np.log(p_yneg)
    c = term1+term2+term3+term4+term5+term6

    return c
```

In [24]:
```
# DO NOT DELETE THIS CELL!
c = calc_c(
    np.array([-0.14820707, -0.037901126], dtype=np.float32),
    np.array([[8.44226725, 2.01781502], [2.01781502, 0.95332618]], dtype=
```

```
    np.array([0.03403238, 0.16727877], dtype=np.float32),
    np.array([[ 0.19860714, -0.16106351], [-0.16106351, 12.55336585]], dt
    0.5,
    0.5
)
assert isinstance(c, float), "Parameter c is not a float!"
assert np.isclose(c, -0.2355323516530916, atol=1e-4), "Parameter c is not
```

In [25]:
```
def calc_g_opt(A: np.ndarray, b: np.ndarray, c: float, gridpoints: np.nda
    """Compute the optimal classification function `g_opt` using the para

    This function applies the classification function `g_hat` to each poi
    Afterwards the sign function is applied using `np.sign` resulting in

    Parameters
    ----------
    A : (K, K) np.ndarray
        A matrix representing quadratic terms in the classification funct
    b : (K,) np.ndarray
        A vector representing linear terms in the classification function
    c : float
        A scalar bias term in the classification function `g_hat`.
    gridpoints : (N, K) np.ndarray
        Array containing the points to which `g_hat` should be applied.

    Returns
    -------
    classification : (N,) np.ndarray
        A 1D array containing the +1 and -1 values of the classification
    """
    # YOUR CODE HERE
    g_hat = np.zeros(gridpoints.shape[0])
    for i, x in enumerate(gridpoints):
        term1 = -0.5 * (x.T @ A @ x)
        term2 = b.T @ x
        g_hat[i] = term1 + term2 + c

    g_opt = np.sign(g_hat)

    return g_opt
```

In [26]:
```
# DO NOT DELETE THIS CELL!
X1, X2 = np.mgrid[-11:11:500j, -11:11:500j]
X1, X2 = X1.ravel(), X2.ravel()
gridpoints = np.c_[X1, X2]
A = np.array([[-4.8482757, -0.5726957], [-0.5726957,  2.0424585]], dtype=
b = np.array([-0.2003752 , -0.02094711], dtype=np.float32)
c = -0.2355323516530916

g_opt = calc_g_opt(A, b, c, gridpoints)
assert g_opt.shape == X1.shape, "The output has a wrong shape!"
assert np.isin(g_opt, [1, -1]).all(), "Some values in g_opt are unequal t
```

In [27]:
```
# Nothing to do here, just run the cell.
print("gridponts.shape =", gridpoints.shape, "\n")
print(30*"=")
print("g_opt.shape =", g_opt.shape, "\n")
print(30*"=")
# Print the values of A, b and c that you have calculated with their resp
```

```
print("A = ", A)
print("A.shape = ", A.shape, "\n")
print(30*"=")
print("b = ", b)
print("b.shape = ", b.shape, "\n")
print(30*"=")
print("c = ", c)
```

```
gridponts.shape = (250000, 2)

==============================
g_opt.shape = (250000,)

==============================
A =  [[-4.8482757 -0.5726957]
 [-0.5726957  2.0424585]]
A.shape =  (2, 2)

==============================
b =  [-0.2003752  -0.02094711]
b.shape =  (2,)

==============================
c =  -0.2355323516530916
```

Plot 2.2 (8 Points):

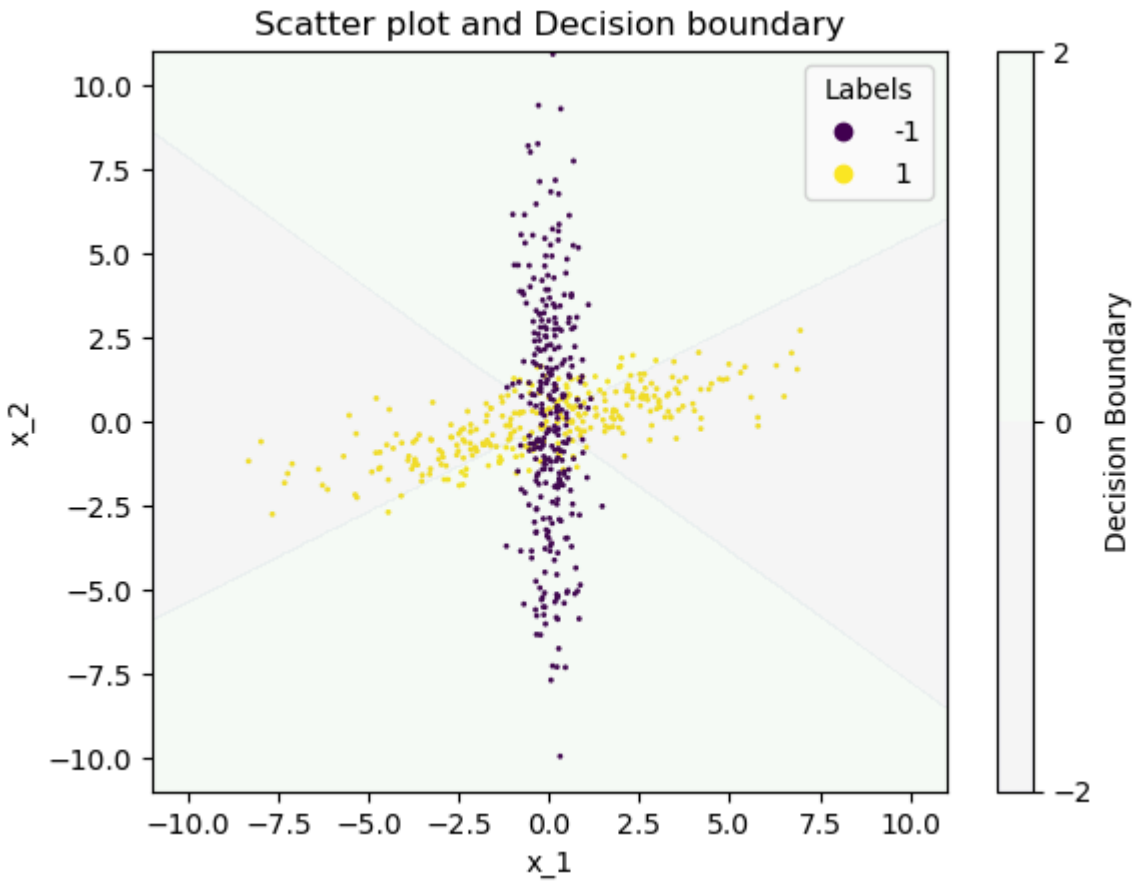
In [81]:
```
def scatter_plot_g_opt(X: np.ndarray, y: np.ndarray, X1: np.ndarray, X2:
    """Creates a scatter-plot for the dataset X with labels y and the cla

    Parameters
    ----------
    X : (N_data, K) np.ndarray
        The original data.
    y : (N_data,) np.ndarray
        The labels of the data.
    X1: (N_grid,) np.ndarray
        The grid values for the x-axis.
    X2: (N_grid,) np.ndarray
        The grid values for the y-axis.
    g_opt : (N_grid,) np.ndarray
        The result of applying the optimal classifier on the grid feature
    """
    # YOUR CODE HERE

    g_optgrid = g_opt.reshape(500,500)
    scatter = plt.scatter(X[:,0], X[:,1], c=y, cmap="viridis", s=1)
    plt.legend(handles=scatter.legend_elements()[0], labels=["-1", "1"],
    plt.contourf(np.unique(X1), np.unique(X2), g_optgrid, levels=0, alpha

    plt.xlabel('x_1')
    plt.ylabel('x_2')
    plt.title('Scatter plot and Decision boundary')
    plt.colorbar(label='Decision Boundary')
    plt.show()
```

In [82]:
```
# Nothing to do here, just run the cell.
scatter_plot_g_opt(X, y, X1, X2, g_opt)
```



Question 2.2 (2 Points):

Answer the following questions about the plot you just created:

a2_) Did the classifier perform well on the task i.e. do the decision boundaries seem to match the classes as plotted in Task 1.1?
b2_) Are datapoints that lie in the middle (i.e. overlapping) region of the two classes less prone to being misclassified compared to data far away from the center?

To answer the question, assign "True" or "False" boolean values to variables in the next cell. For example, if you think that **a2_)** is correct, define a variable `a2_` and set it to `True`, the same applies to **b2_)**. A non-correctly answered question as well as no answer (i.e. answer “None”) yields 0 points for a specific question.
Note: Do not reuse these variable names. They are used for testing.

In [30]:
```
# YOUR CODE HERE
a2_ = True
b2_ = False
```

In [31]:
```
# DO NOT DELETE THIS CELL!
assert a2_ is not None, "Store True/False!"
assert a2_ in [True, False], "Invalid Answer!"
```

In [32]:
```
# DO NOT DELETE THIS CELL!
assert b2_ is not None, "Store True/False!"
assert b2_ in [True, False], "Invalid Answer!"
```

Task 3: Details for bias-variance decomposition for quadratic loss

An explicit formula of the bias variance decomposition for the quadratic loss was mentioned in the lecture. In this task, you will prove this decomposition yourselves. To this end, let us introduce some notation:

$Z_l = (X, \mathbf{y})$ denotes a data matrix of l elements, with X the $(d \times l)$-dimensional feature matrix and $\mathbf{y}$ the l-dimensional (column) label vector. $g(\mathbf{x}_0; \mathbf{w}(Z_l)))$ denotes the model, parametrized by the vector $\mathbf{w}(Z_l)$ trained on Z_l, and the variable y is the label corresponding to a (new) feature vector $\mathbf{x}_0$.

Our object of interest is the expected prediction error (EPE) for $\mathbf{x}_0$ in case of the quadratic loss, i.e.:

$$\mathrm{EPE}(\mathbf{x}_0) = \mathrm{E}_{y|\mathbf{x}_0, Z_l}\big(L_{\mathbf{q}}(y, g(\mathbf{x}_0; \mathbf{w}(Z_l)))\big) = \mathrm{E}_{y|\mathbf{x}_0, Z_l}\big((y - g(\mathbf{x}_0; \mathbf{w}(Z_l)))^2\big)$$

We assume that $y\,|\,\mathbf{x}_0$ and the selection of training samples Z_l are independent which results in the following reformulation of the total expected prediction error:

$$\mathrm{EPE}(\mathbf{x}_0) = \mathrm{E}_{y|\mathbf{x}_0}\Big(\mathrm{E}_{Z_l}\big((y - g(\mathbf{x}_0; \mathbf{w}(Z_l)))^2\big)\Big) \qquad (1)$$

Show that we can obtain the following bias-variance decomposition:

$$\begin{aligned}\mathrm{EPE}(\mathbf{x}_0) = &\mathrm{Var}(y\,|\,\mathbf{x}_0) && (1)\\ &+ \Big(\mathrm{E}_{y|\mathbf{x}_0}(y) - E_{Z_l}\big(g(\mathbf{x}_0; \mathbf{w}(Z_l))\big)\Big)^2 && (2)\quad (2)\\ &+ \mathrm{E}_{Z_l}\Big(\big(g(\mathbf{x}_0; \mathbf{w}(Z_l)) - E_{Z_l}(g(\mathbf{x}_0; \mathbf{w}(Z_l)))\big)^2\Big) && (3)\end{aligned}$$

For your calculation please use the given notation. Follow the steps indicated below.

Calculation 3.1 (5 Points): Expand the Expected Prediction Error.

Expand $\mathrm{EPE}(\mathbf{x}_0)$, i.e. eq. (1) above, and write it as three separate terms.

$$\mathrm{EPE}(\mathbf{x}_0) = \mathrm{E}_{y|\mathbf{x}_0}\left(\mathrm{E}_{Z_l}\left[(y - g(\mathbf{x}_0; \mathbf{w}(Z_l)))^2\right]\right) = \mathrm{E}_{y|\mathbf{x}_0}\left(\mathrm{E}_{Z_l}\left[y^2 - 2yg(\mathbf{x}_0; \mathbf{w}(Z_l)) + g\right.\right.$$

$$\left.\left.\mathrm{E}_{y|\mathbf{x}_0}\left[\mathrm{E}_{Z_l}\left(y^2\right)\right] - 2\mathrm{E}_{y|\mathbf{x}_0}\left[\mathrm{E}_{Z_l}\left(yg(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right] + \mathrm{E}_{y|\mathbf{x}_0}\left[\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))^2\right)\right]\right.\right.$$

Calculation 3.2 (5 Points): Rewrite $\mathrm{Var}(y\,|\,\mathbf{x}_0)$ using expected values.

Write the label variance (unavoidable error), i.e. term 1 in (2), in terms of expectation values.

$$\mathrm{E}_{y|\mathbf{x}_0}\left[\mathrm{E}_{Z_l}\left(y^2\right)\right] = \mathrm{Var}(y \mid \mathbf{x}_0) + \left(\mathrm{E}_{y|\mathbf{x}_0}(y)\right)^2$$

Calculation 3.3 (4 Points): Expand the squared bias.

Expand the squared bias, i.e. term 2 in (2), and write it in three separate terms.

$$\left(\mathrm{E}_{y|\mathbf{x}_0}(y) - \mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right)^2$$

$$= \left(\mathrm{E}_{y|\mathbf{x}_0}(y)\right)^2 - 2\mathrm{E}_{y|\mathbf{x}_0}(y)\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right) + \left(\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right)^2$$

Calculation 3.4 (5 Points): Expand the variance of the model.

Expand model variance, i.e. term 3 in (2), into three terms, and then simplify it to only two terms.

Eventually, show that adding up your results from 3.2., 3.3., and 3.4. leads to your results from 3.1. That concludes the proof.

$$= \left(\mathrm{E}_{y|\mathbf{x}_0}(y)\right)^2 - 2\mathrm{E}_{y|\mathbf{x}_0}(y)\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right) + \left(\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right)^2$$

Expanding the average of the Variance of preds, term 3 in (2):

$$\mathrm{E}_{Z_l}\left[\left(g(\mathbf{x}_0; \mathbf{w}(Z_l)) - \mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right)^2\right]$$

$$= \mathrm{E}_{Z_l}\left[g(\mathbf{x}_0; \mathbf{w}(Z_l))^2\right] - 2\mathrm{E}_{Z_l}\left[g(\mathbf{x}_0; \mathbf{w}(Z_l))\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right] + \left(\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right)$$

Adding 3.2,3.3,3.4:

$\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)$ is a constant w.r.t E_{Z_l}, the term in the middle equals zero:

$$= \mathrm{E}_{Z_l}\left[g(\mathbf{x}_0; \mathbf{w}(Z_l))^2\right] - \left(\mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right)^2$$

We get:

$$\mathrm{EPE}(\mathbf{x}_0) = \mathrm{Var}(y \mid \mathbf{x}_0) + \left(\mathrm{E}_{y|\mathbf{x}_0}(y) - \mathrm{E}_{Z_l}\left(g(\mathbf{x}_0; \mathbf{w}(Z_l))\right)\right)^2 + \mathrm{E}_{Z_l}\left(\left(g(\mathbf{x}_0; \mathbf{w}(Z_l)) -\right.\right.$$

Task 4: Bias-variance decomposition for regression

We intend to perform polynomial regression to illustrate the bias-variance decomposition for the regression task described before. To this end, perform the following steps:

- **Question 4.1**:
  Consider the following one-dimensional regression task: inputs x are sampled from the uniform distribution in $[-1, 3] \subset \mathbb{R}$ and targets y are given as

$$f(x) = 0.5\,x^4 + 2\,x^3 - 8\,x^2$$
$$y = f(x) + \varepsilon,$$

  where ε is independent normally distributed noise with mean $\mu = 0$ and variance $\sigma^2 = 4$.

  Now, answer the given question.

- **Code 4.2**:
  - If a seed is given as function parameter, then set the seed using np.random.seed.
  - Define the function `f(x)`, using the definition in **Question 4.1**.
  - Implement the function `create_X_train` which should return S training sets with N samples in the form of a `np.ndarray`. The x-values are uniformly sampled from `x_min` to `x_max`. Important: Use the function np.random.uniform to complete this task!
  - Implement the function `create_y_train` which generates the training y-values according to **Question 4.1**, i.e. with Gaussian noise. Define the function for general mean `mu` and standard deviation `std`. Important: Use the function np.random.normal to complete this task!
  - Below, we provide the code for a function `pol_reg_pred` that trains a polynomial regression model with degree m on a given training set and returns the prediction for a given test set (uniformly sampled x values without labels). Use this to implement the function `bias_var` that estimates for each degree $m = 1, \ldots, M$ the squared bias and the variance from the predictions for each of the S training sets at x_{test} and returns them in the `np.ndarrays` sqbias and variance. Each of these two arrays should then only contain M elements.
- **Plot 4.3**:
  Utilize the function `pol_reg_pred` to produce a *single* plot that simultaneously visualizes the training data as dots (plot only the *first* instance of the S training sets, i.e. `X_train[0]`) and the corresponding models for $m = 1, 4, 11$. Don't forget to label the axes and give your plot a fitting title.

- **Plot 4.4**:
  Finally, visualize your results in another *single* plot where the dependence of **(i) the unavoidable error**, **(ii) the squared bias**, **(iii) the model variance**, and **(iv) the total EPE** is shown versus m. Again, don't forget to label the axes and give your plot a fitting title as well as plot a legend. Feel free to plot lines to guide the eye, although the horizontal axis is discrete.

- **Question 4.4**:
  Answer some questions about **Plot 4.4**.

Question 4.1 (6 Points):

What are $E(y|x_0)$ and the unavoidable error $\mathrm{Var}(y|x_0)$ for a fixed x_0 in the setting given above?

a3_) $E(y|x_0) = 0.5\,\sigma^4 + 2\,\sigma^3 - 8\,\sigma^2$ and $\mathrm{Var}(y|x_0) = x_0^2$.
b3_) $E(y|x_0) = 0.5\,\sigma^4 + 2\,\sigma^3 - 8\,\sigma^2$ and $\mathrm{Var}(y|x_0) = \sigma^2$.
c3_) $E(y|x_0) = 0.5\,\sigma^4 + 2\,\sigma^3 - 8\,\sigma^2$ and $\mathrm{Var}(y|x_0) = 2\sigma^2$.
d3_) $E(y|x_0) = 0.5\,x_0^4 + 2\,x_0^3 - 8\,x_0^2$ and $\mathrm{Var}(y|x_0) = 2\sigma^2$.
e3_) $E(y|x_0) = 0.5\,x_0^4 + 2\,x_0^3 - 8\,x_0^2$ and $\mathrm{Var}(y|x_0) = \sigma^2$.
f3_) $E(y|x_0) = 0.5\,x_0^4 + 2\,x_0^3 - 8\,x_0^2$ and $\mathrm{Var}(y|x_0) = 0.5\,x_0^4 + 2\,x_0^3 - 8\,x_0^2 + \sigma^2$.

To answer the question, assign "True" or "False" boolean values to variables in the next cell. For example, if you think that **a3_)** is correct, define a variable `a3_` and set it to `True`, the same applies to **b3_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer “None”) yields 0 points for a specific question.

Note: Do not reuse these variable names. They are used for testing.

In [33]:
```
# YOUR CODE HERE
a3_ = False
b3_ = False
c3_ = False
d3_ = False
e3_ = True
f3_ = False
```

In [34]:
```
# DO NOT DELETE THIS CELL!
assert a3_ is not None, "Store True/False!"
assert a3_ in [True, False], "Invalid Answer!"
```

In [35]:
```
# DO NOT DELETE THIS CELL!
assert b3_ is not None, "Store True/False!"
assert b3_ in [True, False], "Invalid Answer!"
```

In [36]:
```
# DO NOT DELETE THIS CELL!
assert c3_ is not None, "Store True/False!"
assert c3_ in [True, False], "Invalid Answer!"
```

In [37]:
```
# DO NOT DELETE THIS CELL!
assert d3_ is not None, "Store True/False!"
assert d3_ in [True, False], "Invalid Answer!"
```

In [38]:
```
# DO NOT DELETE THIS CELL!
assert e3_ is not None, "Store True/False!"
assert e3_ in [True, False], "Invalid Answer!"
```

In [39]:
```
# DO NOT DELETE THIS CELL!
assert f3_ is not None, "Store True/False!"
assert f3_ in [True, False], "Invalid Answer!"
```

4.2 Code (15 points):

In [ ]:
```
# Nothing to do here, just run the cell.
def pol_reg_pred(X_train: np.ndarray, y_train: np.ndarray, x_test: np.nda
    """Train a polynomial regression model of specified degree and return

    This function fits a polynomial regression model to the provided trai
    with degree `m`, then uses this model to predict outputs for the test

    Parameters
    ----------
    X_train : (N,) np.ndarray
        Vector of training samples.
    y_train : (N,) np.ndarray
        Vector of training labels.
    x_test : np.ndarray
        Vector of test samples for which predictions will be made.
    m : int
        Degree of the polynomial for regression.
    seed : int
        Seed for reproducibility.

    Returns
    -------
    prediction : np.ndarray
        Predictions for the `x_test` samples.
    """
    np.random.seed(seed)

    poly_reg = PolynomialFeatures(m)

    X_poly_train = poly_reg.fit_transform(X_train.reshape(-1, 1))
    X_poly_test= poly_reg.fit_transform(x_test.reshape(-1, 1))

    lin_reg = LinearRegression()

    lin_reg.fit(X_poly_train, y_train)
    y_pred = lin_reg.predict(X_poly_test)

    return y_pred
```

In [41]:
```
def f(x: np.ndarray) -> np.ndarray:
    """Implementation of the polynomial from Question 4.1.

    Parameters
    ----------
    x : (N,) np.ndarray
        The input of the function.

    Returns
    -------
    output : (N,) np.ndarray
        The output of the function.
    """
    # YOUR CODE HERE
    return 0.5*(x**4) + 2*(x**3) - 8*(x**2)
```

In [42]:
```
# DO NOT DELETE THIS CELL!
x = np.array([0.5, 2, 3, -4])
out = f(x)
assert isinstance(out, np.ndarray), "Output of function f is not a np.nda
assert out.shape == x.shape, "Output of funcfion f has a wrong shape!"
np.testing.assert_array_equal(out, np.array([-1.71875, -8., 22.5, -128.])
```

In [ ]:
```
def create_X_train(S: int, N: int, x_min: float, x_max: float, seed: int)
    """Generate `S` training sets, each containing `M` samples, within a
    The function creates an array of shape (S, N) where each element is a

    Important: Use the function `np.random.uniform` to generate the data

    Parameters
    ----------
    S : int
        Number of training sets to create.
    N : int
        Number of samples per training set.
    x_min : float
        Lower bound of the sampling interval.
    x_max : float
        Upper bound of the sampling interval.
    seed : int
        Seed for reproducibility.

    Returns
    -------
    X_train : (S, N) np.ndarray
        A np.ndarray containing the generated sets of samples.
    """
    # YOUR CODE HERE

    np.random.seed(seed)

    X_train = np.random.uniform(x_min,x_max,(S,N))

    return X_train
```

In [44]:
```
# DO NOT DELETE THIS CELL!
S = 300
N = 25
x_min = -1
x_max = 3
seed = 66

X_train = create_X_train(S, N, x_min, x_max, seed)
assert isinstance(X_train, np.ndarray), "X_train is not a np.ndarray!"
assert X_train.shape == (S, N), "The output shape of X_train is incorrect
```

In [ ]:
```
def create_y_train(X_train: np.ndarray, mu: float, std: float, f: callabl
    """Generate labels from training data using a function `f` with added

    This function applies `f` to the training set `X_train` to generate l
    Gaussian noise with specified mean `mu` and standard deviation `std`
    A seed is used for reproducibility.

    Important: Use `np.random.seed` to set the seed and np.random.normal
```

```
    Parameters
    ----------
    X_train : (S, N) np.ndarray
        The training data set to which the function `f` will be applied.
    mu : float
        Mean of the Gaussian noise.
    std : float
        Standard deviation of the Gaussian noise.
    f : callable
        A function to apply to `X_train` to generate labels.
    seed : int
        Random seed for reproducibility.

    Returns
    -------
    y_train : (S, N) np.ndarray
        An array containing the generated labels with added noise.
    """
    # YOUR CODE HERE
    np.random.seed(seed)

    y_train = f(X_train)
    noise = np.random.normal(mu,std,y_train.shape)
    y_train = y_train+noise

    return y_train
```

In [46]:
```
# DO NOT DELETE THIS CELL!
seed = 66
mu = 0
std = 2
ref_X_train = np.arange(10).reshape(2, -1)

ref_y_train = create_y_train(ref_X_train, mu, std, lambda x: x**2, seed)
assert isinstance(ref_y_train, np.ndarray), "y_train is not a np.ndarray!
assert ref_y_train.shape == ref_X_train.shape, "y_train has a wrong shape
np.testing.assert_array_almost_equal(ref_y_train, np.array([[2.83122873,
```

In [ ]:
```
def bias_var(X_train: np.ndarray, y_train: np.ndarray, x_test: np.ndarray
    """Compute model bias and variance for polynomial regression models o

    This function estimates the squared bias and variance of polynomial r
    of degrees ranging from 1 to `M` by sampling training sets, fitting e
    and evaluating each at a specified test set `x_test`.

    Important: Pass the seed to `pol_reg_pred` and use this function
    for computing the predictions of the polynomial regression models!

    Parameters
    ----------
    X_train : (S, N) np.ndarray
        Array of training data sets, where each element is an array of sa
    y_train : (S, N) np.ndarray
        Array of training label sets, where each element is an array of l
    x_test : np.ndarray
        A 1D test data set.
    M : int
        The maximum polynomial degree to evaluate.
```

```
    f : callable
        The true function to compare the predictions against.
    seed : int
        Random seed for reproducibility.

    Returns
    -------
    tuple of np.ndarray
        - sqbias : (M,) np.ndarray
            Array of squared bias values for polynomial degrees from 1 to
        - variance : (M,) np.ndarray
            Array of variance values for polynomial degrees from 1 to `M`
    """
    # YOUR CODE HERE
    np.random.seed(seed)

    S = X_train.shape[0]
    sqbias = np.zeros(M)
    variance = np.zeros(M)

    for m in range(1, M+1):
        preds = np.zeros((S, len(x_test)))
        for i in range(S):
            ypred = pol_reg_pred(X_train[i], y_train[i], x_test, m, seed)
            preds[i] = ypred

        ypred_mean = np.mean(preds,axis=0)

        bias = np.mean((f(x_test)-ypred_mean)**2)
        sqbias[m-1] = bias

        var = np.mean((preds-ypred_mean)**2, axis=0)
        variance[m-1] = np.mean(var)

    return sqbias, variance
```

In [48]:
```
# DO NOT DELETE THIS CELL!
seed = 66
M = 11
ref_X_train = np.arange(10).reshape(2, -1)
ref_y_train = np.arange(10, 20).reshape(2, -1)
ref_x_test = np.arange(5)

ref_sqbias, _ = bias_var(ref_X_train, ref_y_train, ref_x_test, M, lambda
assert isinstance(ref_sqbias, np.ndarray), "The returned squared bias is
assert ref_sqbias.shape == (M,), "The returned squared bias has a wrong s
np.testing.assert_array_almost_equal(ref_sqbias, np.array([56.8, 56.8, 56
```

In [49]:
```
# DO NOT DELETE THIS CELL!
seed = 66
M = 11
ref_X_train = np.arange(10).reshape(2, -1)
ref_y_train = np.arange(10, 20).reshape(2, -1)
ref_x_test = np.arange(5)

_, ref_var = bias_var(ref_X_train, ref_y_train, ref_x_test, M, lambda x:
assert isinstance(ref_var, np.ndarray), "The returned variance is not a n
assert ref_var.shape == (M,), "The returned variance has a wrong shape!"
np.testing.assert_array_almost_equal(ref_var, np.array([0.0000e+00, 2.524
```

In [50]:
```
# Nothing to do here, just run the cell.
seed = 12
S = 300
N = 25
M = 11
x_min = -1
x_max = 3
mu = 0
sigma_squared = 4
std = np.sqrt(sigma_squared)
x_test = np.array([1.7])

X_train = create_X_train(S,N,x_min,x_max,seed)
y_train = create_y_train(X_train,mu,std,f,seed)
sqbias, variance = bias_var(X_train,y_train,x_test,M,f,seed)
print("Shapes of X and y: \n",X_train.shape,y_train.shape)
print("\nSquared Bias over m: \n", sqbias)
print("\nVariance over m: \n", variance)
```

```
Shapes of X and y:
 (300, 25) (300, 25)

Squared Bias over m:
 [5.50461692e+01 2.58892159e+01 6.39322745e-03 1.51847647e-06
 8.31015750e-05 3.52859200e-03 9.14787903e-03 1.74217969e-03
 1.61017488e-02 1.01851959e-03 4.57567286e-02]

Variance over m:
 [ 4.43612857  2.30715582  0.6861143   0.72331817  1.03163253  1.28427733
  1.79658367  2.78252631  2.84032616  6.33845148 19.35307884]
```

Plot 4.3 (5 Points):

In [51]:
```
# Nothing to do here, just run the cell.
x_test = np.arange(x_min, x_max, 0.01)
```

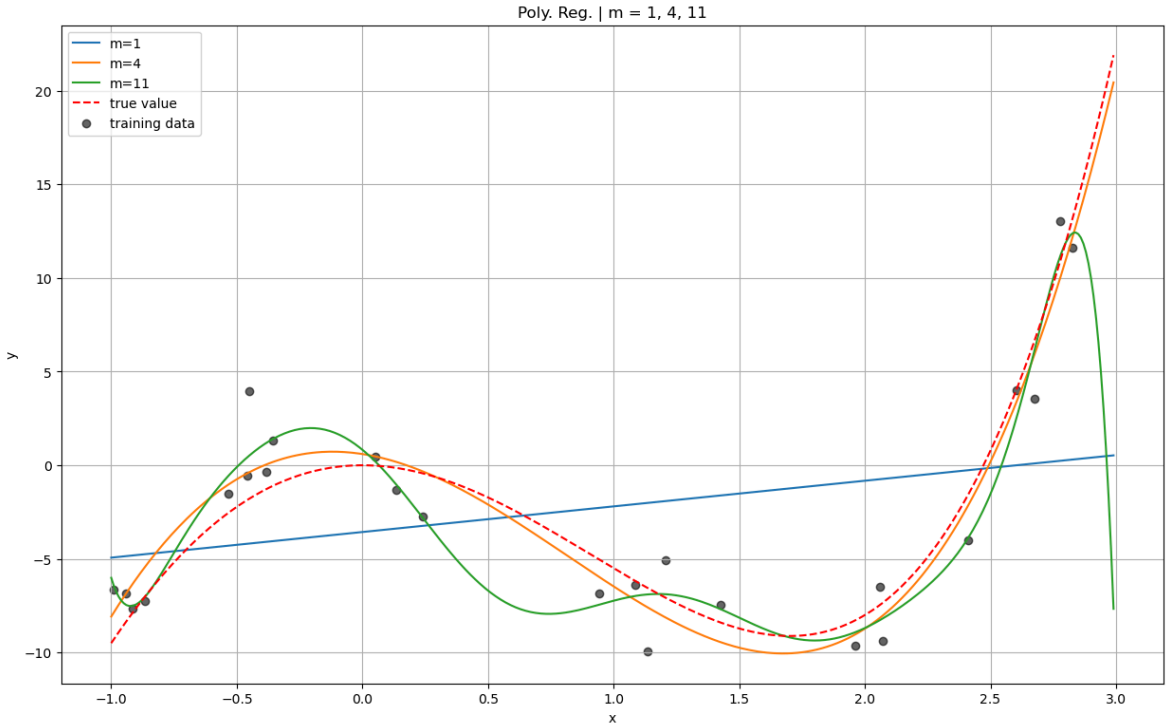
In [52]:
```
def plot_regression(X_train: np.ndarray, y_train: np.ndarray, x_test: np.
    """Creates a plot for the first set of training data and the correspo

    Important: Pass the seed to `pol_reg_pred` and use this function
    for computing the predictions of the polynomial regression models!

    Parameters
    ----------
    X_train : (S, N) np.ndarray
        Sets of training data.
    y_train : (S, N) np.ndarray
        Sets of training labels.
    x_test : np.ndarray
        Vector of test data.
    """
    # YOUR CODE HERE
    plt.figure(figsize=(15, 9))
    for m in [1, 4, 11]:
        y_pred = pol_reg_pred(X_train[0], y_train[0], x_test, m, seed)
        plt.plot(x_test, y_pred, label=f"m={m}")
    y_true = f(x_test)
```

```
    plt.plot(x_test, y_true, label="true value", color="red", linestyle="
    plt.scatter(X_train[0], y_train[0], color='black', label="training da
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('Poly. Reg. | m = 1, 4, 11')
    plt.legend()
    plt.grid(True)
    plt.show()
```

In [53]:
```
# Nothing to do here, just run the cell.
plot_regression(X_train, y_train, x_test, seed)
```



Plot 4.4 (5 Points):

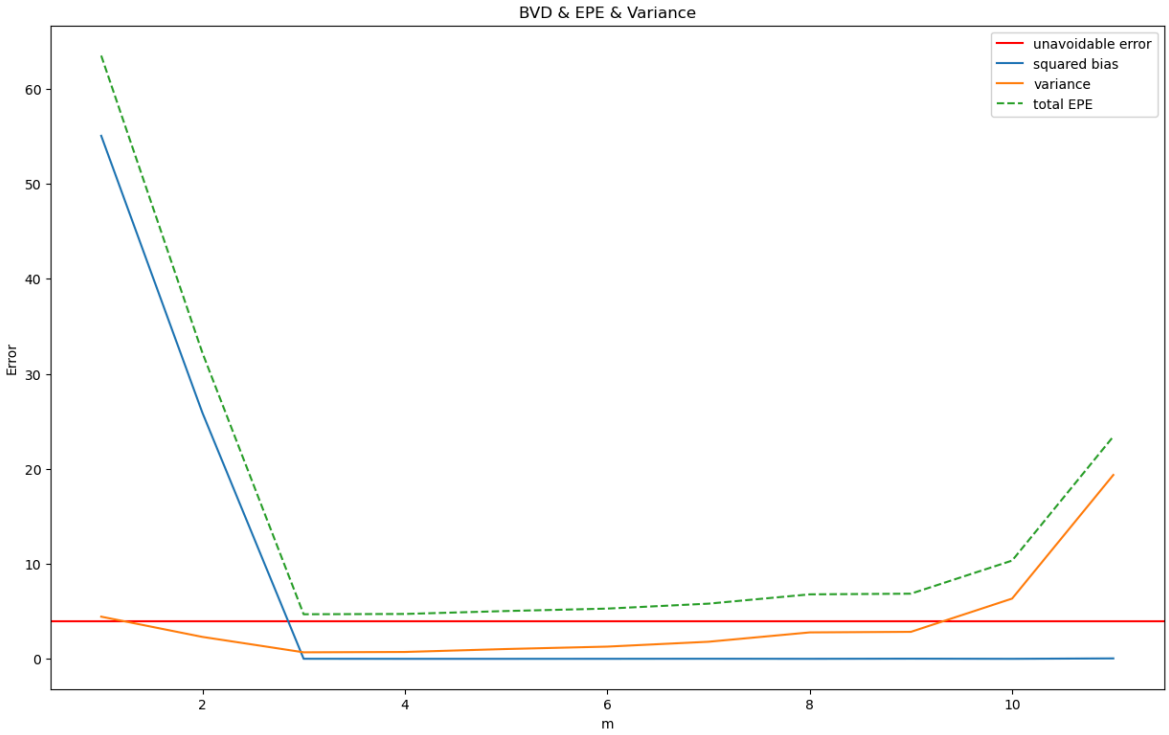
In [54]:
```
def plot_combination(M: int, sigma: float, bias: np.ndarray, var: np.ndar
    """Creates a plot for the unavoidable error, bias, variance, and Expe

    Parameters
    ----------
    M : int
        The maximum polynomial degree for which to plot the error compone
    sigma : float
        The unavoidable error level, representing noise in the data.
    bias : np.ndarray
        Array of squared bias values for polynomial degrees in [1, M].
    var : np.ndarray
        Array of variance values for polynomial degrees in [1, M].
    """
    # YOUR CODE HERE

    EPE = sigma_squared + sqbias + var
    plt.figure(figsize=(15, 9))
    plt.axhline(y=sigma_squared, color="red", label="unavoidable error")
    plt.plot(range(1, M+1), bias, label="squared bias")
    plt.plot(range(1, M+1), var, label="variance")
    plt.plot(range(1, M+1), EPE, label="total EPE", ls="dashed")
```

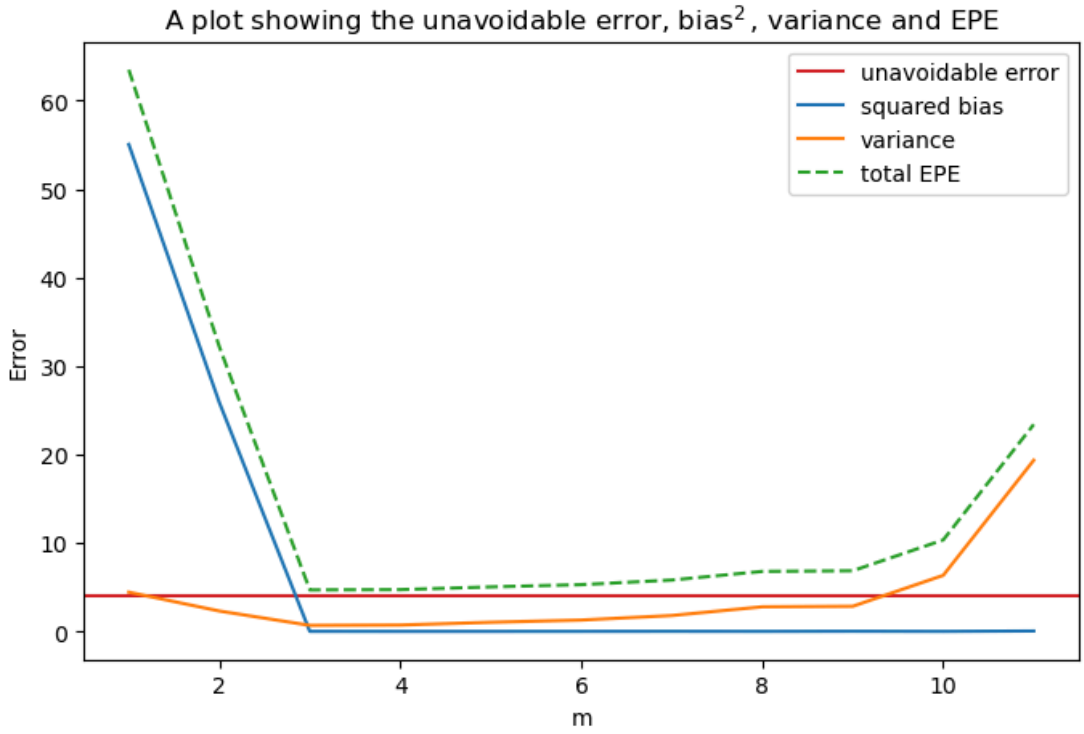
```
    plt.xlabel("m")
    plt.ylabel("Error")
    plt.title("BVD & EPE & Variance")
    plt.legend()
    plt.show()
```

In [55]:
```
# Nothing to do here, just run the cell.
plot_combination(M, sigma_squared, sqbias, variance)
```



Question 4.4 (5 Points):

If you did the previous task correctly, the resulting plot should have many similarities or should be identical to the following plot:



What observations can you make from this plot? Answer which of the following statements are correct (several may be correct):

a4_) The variance is low for models which are too complex, i.e. $m \leq 2$.
b4_) For appropriate complexity, i.e. $3 \leq m \leq 6$, both model variance and bias are low, which can indicate good generalization abilities.
c4_) As the model becomes too complex, i.e. $m \geq 9$, the variance increases again while the bias stays low. This is an indication for overfitting.
d4_) For models with $m \geq 9$, the variance is high (i.e. significantly larger than 0) because the independent noise has zero mean and high individual biases cancel in expectation.
e4_) For models with $m \geq 9$, the bias is still low (i.e. close to 0) because the independent noise has zero mean and high individual biases cancel in expectation.

To answer the question, assign "True" or "False" boolean values to variables in the next cell. For example, if you think that **a4_)** is correct, define a variable `a4_` and set it to `True`, the same applies to **b4_)** and the other statements. A non-correctly answered statement as well as no answer (i.e. answer “None”) yields 0 points for a specific statement.
Note: Do not reuse these variable names. They are used for testing.

In [56]:
```
# YOUR CODE HERE
a4_ = False
b4_ = True
c4_ = True
d4_ = False
e4_ = False
```

In [57]:
```
# DO NOT DELETE THIS CELL!
assert a4_ is not None, "Store True/False!"
```

```
assert a4_ in [True, False], "Invalid Answer!"
```

In [58]:
```
# DO NOT DELETE THIS CELL!
assert b4_ is not None, "Store True/False!"
assert b4_ in [True, False], "Invalid Answer!"
```

In [59]:
```
# DO NOT DELETE THIS CELL!
assert c4_ is not None, "Store True/False!"
assert c4_ in [True, False], "Invalid Answer!"
```

In [60]:
```
# DO NOT DELETE THIS CELL!
assert d4_ is not None, "Store True/False!"
assert d4_ in [True, False], "Invalid Answer!"
```

In [61]:
```
# DO NOT DELETE THIS CELL!
assert e4_ is not None, "Store True/False!"
assert e4_ in [True, False], "Invalid Answer!"
```

Task 5: Evaluation metrics for imbalanced data sets

Consider a classifier with discriminant function $\bar{g}$. For a given labeled data set, the following results are obtained:

y	$\bar{g}(x)$
$+1$	0.93
$+1$	0.51
-1	0.48
-1	0.13
$+1$	0.02
-1	-0.11
-1	-0.25
-1	-0.37
$+1$	-0.41
-1	-1.68
$+1$	-2.23
$+1$	-3.41

- **Code 5.1**:
  - Compute the confusion matrix using the usual $\theta = 0$ threshold.
  - Complete the function `evaluation_metrics` to calculate the following evaluation measures: TPR, TNR, FPR, FNR, ACC, BACC, PREC, and F_1.

Let's say we have a population of 1000 people and we know that 50 are infected with the corona virus.

- **Code 5.2**: Assume that the population is tested with an assay that has a certain specificity and sensitivity.

- (1) What is the probability p_1 that a person is *not* infected if they are diagnosed as ill by the test?
- (2) What is the probability p_2 that a person is infected if they are diagnosed as healthy by the test?

Write a function that returns both values. Then check your calculation using specificity of 90% and sensitivity of 95%.

Important: Round your result to 4 decimal points, i.e. 0.9871 if it is 98.71%.

Code 5.1 (8 Points):

- **Reminder:** Confusion Matrix structure:

	$g(x) = +1$	$g(x) = -1$
$y = +1$	TP	FN
$y = -1$	FP	TN

Define 4 variables `TP_` , `FN_` , `FP_` and `TN_` and assign the correct number assuming the usual $\theta = 0$ threshold.

In [62]:
```
# YOUR CODE HERE
TP_ = 3
FN_ = 3
FP_ = 2
TN_ = 4
```

In [63]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(TP_, int), "The number of TP is not an integer!"
```

In [64]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(FN_, int), "The number of FN is not an integer!"
```

In [65]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(FP_, int), "The number of FP is not an integer!"
```

In [66]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(TN_, int), "The number of TN is not an integer!"
```

In [67]:
```
def evaluation_meatrics(TP: int, TN: int, FP: int, FN: int) -> tuple[floa
    """Calculate evaluation metrics for a binary classification model.

    This function computes common evaluation metrics, including True Posi
    True Negative Rate (TNR), False Positive Rate (FPR), False Negative R
    Accuracy (ACC), Balanced Accuracy (BACC), Precision (PREC), and F1 Sc
    based on the counts of true positives, true negatives, false positive

    Parameters
    ----------
    TP : int
        Number of true positive cases.
    TN : int
```

```
        Number of true negative cases.
    FP : int
        Number of false positive cases.
    FN : int
        Number of false negative cases.

    Returns
    -------
    tuple of float
        - TPR : float
            True Positive Rate, or Recall.
        - TNR : float
            True Negative Rate, or Specificity.
        - FPR : float
            False Positive Rate.
        - FNR : float
            False Negative Rate.
        - ACC : float
            Accuracy of the model.
        - BACC : float
            Balanced Accuracy, the average of TPR and TNR.
        - PREC : float
            Precision of the model.
        - F1 : float
            F1 Score, the harmonic mean of Precision and Recall
    """
    # YOUR CODE HERE
    TPR = TP / (TP+FN)
    TNR = TN / (TN+FP)
    FPR = FP / (TN+FP)
    FNR = FN / (TP+FN)
    ACC = (TP+TN) / (TP+TN+FP+FN)
    BACC = (TPR+TNR) / 2
    PREC = TP / (TP+FP)
    F1 = 2 * ((PREC*TPR) / (PREC+TPR))

    return TPR, TNR, FPR, FNR, ACC, BACC, PREC, F1
```

In [68]:
```
# DO NOT DELETE THIS CELL!
TPR_, TNR_, FPR_, FNR_, ACC_, BACC_, PREC_, F1_ = evaluation_meatrics(TP_
assert isinstance(TPR_, float), "The TPR is not a float!"
assert TPR_ == 0.5, "The TPR is wrong!"
```

In [69]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(TNR_, float), "The TNR is not a float!"
assert TNR_ == 2/3, "The TNR is wrong!"
```

In [70]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(FPR_, float), "The FPR is not a float!"
assert FPR_ == 1/3, "The FPR is wrong!"
```

In [71]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(FNR_, float), "The FNR is not a float!"
assert FNR_ == 0.5, "The FNR is wrong!"
```

In [72]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(ACC_, float), "The ACC is not a float!"
assert round(ACC_, 3) == 0.583, "The ACC is wrong!"
```

In [73]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(BACC_, float), "The BACC is not a float!"
assert round(BACC_, 3) == 0.583, "The BACC is wrong!"
```

In [74]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(PREC_, float), "The PREC is not a float!"
assert PREC_ == 0.6, "The PREC is wrong!"
```

In [75]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(F1_, float), "The F1 is not a float!"
assert F1_ == 6/11, "The F1 is wrong!"
```

In [76]:
```
# Nothing to do here, just run the cell.
print(" TPR = {:.3f}\n TNR = {:.3f}\n FPR = {:.3f}\n FNR = {:.3f}\n ACC =
```

```
 TPR = 0.500
 TNR = 0.667
 FPR = 0.333
 FNR = 0.500
 ACC = 0.583
BACC = 0.583
PREC = 0.600
  F1 = 0.545
```

Calculation 5.2 (7 Points):

In [77]:
```
def calc_prob(spec: float, sens: float, pop: int, inf: int) -> tuple[floa
    """Calculate the rounded probabilities of false positives and false n

    This function computes two probabilities:
    - `p1`: the probability of a false positive given a positive test res
    - `p2`: the probability of a false negative given a negative test res

    Parameters
    ----------
    spec : float
        Specificity of the test (True Negative Rate).
    sens : float
        Sensitivity of the test (True Positive Rate).
    pop : int
        Total population size.
    inf : int
        Number of infected individuals in the population.

    Returns
    -------
    tuple of float
        - p1 : float
            Probability of a false positive given a positive test result,
        - p2 : float
            Probability of a false negative given a negative test result,
    """
    # YOUR CODE HERE
    prev = inf / pop
    pos = (prev * sens) + ((1-prev) * (1-spec))
    neg = (prev * (1-sens)) + ((1-prev) * spec)
    p1 = round(((1-prev) * (1-spec)) / pos,4)
    p2 = round((prev * (1-sens)) / neg,4)
```

```
    return p1, p2
```

In [78]:
```
# DO NOT DELETE THIS CELL!
p1, _ = calc_prob(0.90, 0.95, 1000, 50)
assert isinstance(p1, float), "The probability of a false positive is not
assert p1 == 0.6667, "The probability of a false positive is not correct!
```

In [79]:
```
# DO NOT DELETE THIS CELL!
_, p2 = calc_prob(0.90, 0.95, 1000, 50)
assert isinstance(p2, float), "The probability of a false negative is not
assert p2 == 0.0029, "The probability of a false negative is not correct!
```

In [80]:
```
print("The probability that a person who is tested positive is in fact no
print("The probability that a person who is tested negative is in fact in
```

```
The probability that a person who is tested positive is in fact not infect
ed is 0.6667
The probability that a person who is tested negative is in fact infected i
s 0.0029
```